# Phishing Shield AI — Complete Tech Stack, Machine Learning & Cybersecurity Guide

### Clear Technical Documentation: System Architecture, AI Models, and Threat Defenses

**Author / System Architect:** Ram Narayan | **Framework:** Python FastAPI 3.11 + React 18 + Scikit-Learn 1.5 | **Date:** October 2026

## 1. What is Phishing Shield AI?

**Phishing Shield AI** is an AI-powered cybersecurity system designed to protect users from online scams, fake websites, and fraudulent messages in real time.

When a user submits an email, SMS, WhatsApp message, or website link, the system analyzes the content and instantly returns:

- **Risk Score (0 to 100)**: Quantitative threat score.
- **Verdict**: **SAFE** (0–39), **SUSPICIOUS** (40–69), or **CRITICAL / HIGH RISK** (70–100).
- **Explainable Evidence**: Bulleted reasons explaining why the item was flagged (e.g., suspicious link, fake bank claim, urgent panic language).

## 2. System Workflow (Step-by-Step Flow)

**Step 1: Inbound Message / Webpage Ingestion**

The user enters text in the React Web Dashboard, or the Chrome Extension scans links on an active webpage.

**Step 2: API Gateway & Threat Perimeter (FastAPI Backend)**

The request reaches the Python FastAPI backend. The payload size is checked (capped at 256 KB) and security headers are enforced.

**Step 3: Multi-Vector Threat Analysis**

The message is split into text, links, phone numbers, and sender addresses. Each part is evaluated by specialized engines:

- **Machine Learning Text Classifier**: Analyzes text semantics and scam vocabulary.
- **URL / Domain Threat Engine**: Scans link structure, typosquatting, and unencrypted protocols.
- **Phone / Callback Threat Engine**: Evaluates international country codes and callback fraud traps.
- **Email Spoofing Engine**: Detects institutional claims sent from free or burner email domains.

**Step 4: Hybrid Risk Arbitration & Secure Storage**

The Risk Engine calculates a weighted score (0–100) and saves the threat fingerprint into MongoDB Atlas using SHA-256 Zero-PII hashing.

## 3. Machine Learning Models Used in the System

Phishing Shield AI uses a **hybrid approach** combining statistical machine learning with deterministic security rules.

### 3.1. Primary NLP Classifier (`model.pkl` + `vectorizer.pkl`)

- **Model Type**: L2-Regularized Logistic Regression with Sublinear TF-IDF Vectorization.
- **How It Works (Step-by-Step)**:

1. **Text Cleansing & Tokenization**: Removes HTML tags, lowercases letters, strips zero-width hidden spaces, and tokenizes text into individual words and pairs of words (unigrams and bigrams).
2. **TF-IDF Feature Extraction (`vectorizer.pkl`)**:

- Calculates how frequently a word appears in the message compared to the entire training corpus.
- Rare scam words (e.g., *"lottery"*, *"suspended"*, *"verify KYC"*, *"account blocked"*) receive high mathematical weights.
- Common neutral words (e.g., *"the"*, *"is"*, *"meeting"*, *"tomorrow"*) receive very low or zero weight.

3. **Logistic Regression Prediction (`model.pkl`)**:

- The vector of word weights is fed into the logistic sigmoid function.
- It calculates the exact probability (from 0.0 to 1.0) that the message is malicious.
- If the probability is high, the model outputs a high base risk score.
- **Why Logistic Regression Was Chosen Over Deep Learning (LLMs)**:
- **Ultra-Fast Speed**: Generates predictions in **under 15 milliseconds**, allowing real-time scanning as users browse.
- **Zero Hallucination**: Pure mathematical classification; never generates false text or invents reasons.
- **Resource Efficiency**: Runs smoothly on lightweight cloud servers (CPU only) without requiring expensive GPU infrastructure.
- **Training Dataset (29,113 Verified Samples)**:
- Aggregated from Kaggle Phishing Email datasets, Enron corporate emails (legitimate ham baseline), UCI SMS spam collection, and Dr. Jose Nazario phishing archives.
- **Model Performance Metrics**:
- **Accuracy**: **96.84%**
- **Precision**: **95.42%** (minimizes false alarms on normal business messages)
- **Recall**: **97.10%** (detects 97 out of 100 real scam messages)
- **ROC-AUC**: **0.9891** (near-perfect discriminative capability)

### 3.2. Smishing & Phone Scam Classifier (`phone_pattern_model.pkl`)

- **Purpose**: Detects mobile text scams and callback fraud traps (*"Your power will be cut tonight! Call our officer immediately"*).
- **Detection Mechanics**:
- Analyzes the phone number structure and country calling code (ITU-T E.164).
- Evaluates digit entropy to detect automated VoIP generated numbers.
- Checks if an institutional keyword (e.g., *"Bank"*, *"Electricity"*, *"Tax"*) is paired with an unofficial 10-digit personal mobile number.

### 3.3. Email Sender Discrepancy Classifier (`email_pattern_model.pkl`)

- **Purpose**: Identifies spoofed emails and Business Email Compromise (BEC).
- **Detection Mechanics**:
- Flags official-sounding display names (e.g., *"State Bank of India Security Team"*) coming from free public webmail addresses (`@gmail.com`, `@yahoo.com`).
- Detects disposable temporary burner domains (`tempmail.com`, `mailinator.com`).
- Inspects high-abuse Top-Level Domains (`.xyz`, `.top`, `.buzz`, `.icu`).

### 3.4. Hybrid Explainability Risk Engine (`risk_engine.py`)

- **Purpose**: Combines signals from all models and rule engines into a single, calibrated 0–100 score.
- **Arbitration Logic**:
- **URL Vector Weight**: If a malicious URL is present, it contributes up to 80% of the threat score.
- **Text ML Weight**: Contributes 40% to 50% to base linguistic evaluation.
- **Compound Escalation**: If both text panic language and a suspicious URL are present simultaneously, an automatic compound amplification penalty (+35 points) is added.
- **High-Confidence Override**: Confirmed brand spoofing or toll-fraud callbacks trigger immediate escalation to **CRITICAL**.

## 4. Key Cybersecurity Concepts Applied in the System

Phishing Shield AI incorporates foundational cybersecurity concepts and threat modeling standards:

### 4.1. Social Engineering & Urgency Exploitation

- **Cyber Concept**: Attackers exploit human psychology—specifically fear, curiosity, or greed—to compel victims into taking rushed actions before thinking critically.
- **System Countermeasure**: The NLP model detects urgent language patterns (*"action required within 2 hours"*, *"account will be permanently deleted"*, *"immediate verification"*) and flags artificial urgency as a threat indicator.

### 4.2. Phishing & Spear Phishing

- **Cyber Concept**: Deceptive communications designed to trick users into revealing sensitive data (passwords, PINs, credit card numbers). Spear phishing targets specific individuals with customized organizational context.
- **System Countermeasure**: Evaluates message semantics and sender identity to flag credential-harvesting triggers regardless of how convincing the message appears.

### 4.3. Smishing (SMS Phishing) & Callback Vishing

- **Cyber Concept**: SMS messages or phone traps instructing victims to dial a fake support number to resolve a manufactured emergency (such as a blocked SIM card or pending utility disconnection).
- **System Countermeasure**: The phone analyzer detects institutional mismatches (legitimate banks never ask customers to call personal mobile numbers) and flags known international cybercrime prefixes.

### 4.4. Typosquatting & Homoglyph Attacks

- **Cyber Concept**: Registering domain names that closely resemble trusted brands (e.g., `paypa1.com`, `rnicrosoft.com`, `goog1e.com`) to mislead victims visually.
- **System Countermeasure**: Computes the **Levenshtein Distance** between extracted URLs and a database of trusted financial and tech brands. Any single-character edit triggers a high-risk typosquatting warning.

### 4.5. Zero-Day Threat Interception

- **Cyber Concept**: Attackers register new domains daily, rendering static blacklists ineffective during the initial hours of a campaign.
- **System Countermeasure**: Analyzes the structural heuristics of the URL (raw IP in hostname, suspicious subdomains, unencrypted HTTP protocol, target keywords like `/login`) to flag malicious links before they appear on commercial blacklists.

### 4.6. Defense in Depth (Layered Security)

- **Cyber Concept**: Multiple defensive mechanisms working together so that if one layer fails, subsequent layers prevent compromise.
- **System Countermeasure**:
- Layer 1: Client Chrome Extension intercepts links in the browser DOM.
- Layer 2: API Gateway enforces request limits and security headers.
- Layer 3: Normalization strips zero-width obfuscation and homoglyphs.
- Layer 4: Machine Learning evaluates text semantics.
- Layer 5: Rule engines check URLs, phone numbers, and sender domains.
- Layer 6: Risk engine correlates all findings into an explainable verdict.

### 4.7. Explainable AI (XAI) in Cybersecurity

- **Cyber Concept**: Security Operations Center (SOC) analysts and end users need to know *why* an alert was triggered rather than trusting an opaque "black-box" prediction.
- **System Countermeasure**: Every scan provides a detailed breakdown of detected threat factors, empowering users to make informed security decisions.

### 4.8. ReDoS (Regular Expression Denial-of-Service) Defense

- **Cyber Concept**: Attackers send carefully crafted repeating text strings designed to cause regular expression engines to enter catastrophic backtracking, freezing the server CPU.

- **System Countermeasure**: The API enforces a strict 256 KB request payload cap via `ContentLengthLimitMiddleware` and uses bounded, non-catastrophic regex patterns.

### 4.9. Zero-PII Cryptographic Storage (Data Privacy & GDPR)

- **Cyber Concept**: Security tools should never become targets for credential leaks or violate privacy regulations by storing private user correspondence.
- **System Countermeasure**: Raw messages are never saved in the database. Instead, a **SHA-256 cryptographic hash** of the message text is generated and used for deduplication, ensuring zero personally identifiable information (PII) is stored.

---

## 5. Complete Technology Stack Matrix

| Layer | Technology | Version | Purpose in Simple Words |
|---|---|---|---|
| **Frontend UI** | React | 18.2.0 | Reactive security dashboard updating in real time |
| **Frontend Tooling** | Vite | 5.0.0 | High-speed frontend compiler and asset bundler |
| **Styling** | Tailwind CSS | 3.3.5 | Dark-mode cybersecurity theme with alert badges |
| **Icons** | Lucide React | 0.292.0 | Lightweight security icons (Shield, Warning, Lock) |
| **Backend Language** | Python | 3.11.9 | Core programming language for API and AI inference |
| **Web Framework** | FastAPI | 0.115.x | High-throughput async REST API framework |
| **ASGI Server** | Uvicorn | 0.32.x | Async web server running the FastAPI backend |
| **Data Validation** | Pydantic | 2.x | Strict schema validation blocking malformed inputs |
| **Machine Learning** | Scikit-Learn | 1.5.x | TF-IDF feature extraction and Logistic Regression |
| **Vectorization** | TF-IDF | Custom | Translates message words into numerical features |
| **Model Persistence** | Joblib | 1.4.x | Fast disk serialization for compiled `.pkl` models |
| **Database** | MongoDB Atlas | 7.0+ | Cloud document database storing threat telemetry |
| **Database Driver** | Motor / PyMongo | 3.6.x | Non-blocking async MongoDB client for Python |
| **Browser Extension** | Chrome MV3 | 3.0 | Real-time DOM link and webpage content scanner |
| **Cloud Hosting** | Render / Vercel | Cloud | Production hosting for backend API and frontend UI |